



Grundlagen der Versionskontrolle & Git

Von der Idee zum strukturierten Zeitprotokoll – ein Einstieg in Versionskontrolle, Git-Installation und erste Schritte mit dem mächtigsten Werkzeug der Softwareentwicklung.

CORE TECH FUNDAMENTALS

Was ist Versionskontrolle?

Das Problem ohne VCS

Dateien wie projekt_final, projekt_final_neu, projekt_final_wirklich_final – chaotisch, fehleranfällig und nicht nachvollziehbar.

Die Lösung: Ein Zeitprotokoll

Ein Versionskontrollsystem verwaltet Änderungen **sauber, nachvollziehbar und strukturiert**. Es beantwortet: **was** geändert wurde, **wann**, **von wem** – und oft auch **warum**, wenn sinnvolle Commit-Nachrichten geschrieben wurden.

Laut der offiziellen Git-Einführung sollen Änderungen historisch nachvollziehbar bleiben, frühere Stände wiederherstellbar sein und Zusammenarbeit kontrolliert möglich werden.

Prinzip 1: Nachvollziehbare Projekt-Historie

Das Herzstück eines Versionskontrollsystems ist die **Historie**. In Git heißt ein gespeicherter Stand **Commit**. Ein Commit enthält:

Dateizustand

Zustand aller verfolgten Dateien zum Zeitpunkt des Commits

Metadaten

Autor, E-Mail-Adresse und Zeitstempel

Commit-Nachricht

Beschreibung der vorgenommenen Änderung

Referenz

Verweis auf vorherige Commits – bildet eine Kette



Prinzip 2: Snapshots statt Dateidifferenzen

Wie Git Daten speichert

Git behandelt Daten konzeptionell als **Snapshots**. Bei einem Commit merkt sich Git nicht nur „Zeile 7 wurde geändert“, sondern speichert den Zustand aller verfolgten Dateien wie einen **Schnappschuss des gesamten Projekts**.

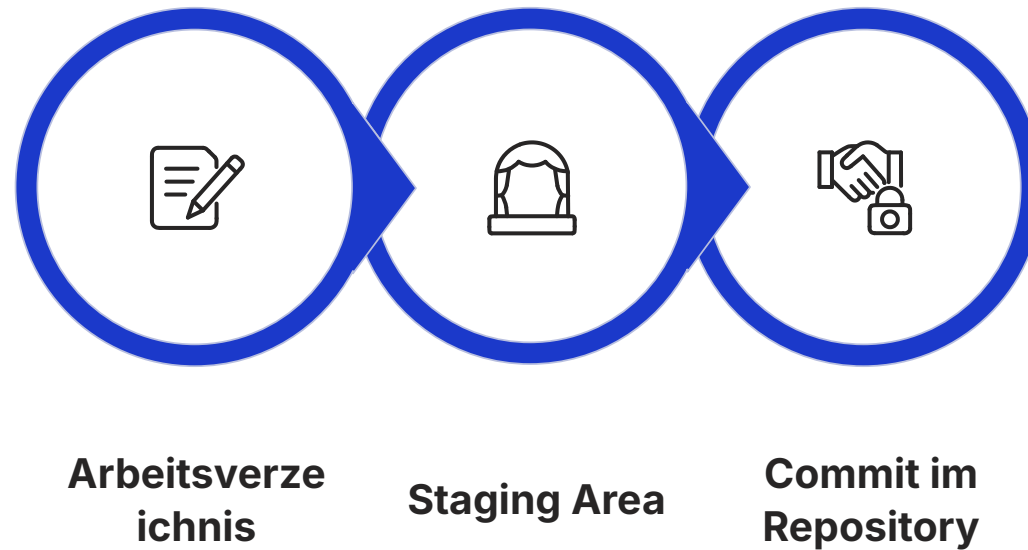
Die Git-Dokumentation erklärt ausdrücklich, dass Git eher in Snapshots als in klassischen zeilenbasierten Deltas denkt.

Warum das wichtig ist

Viele Anfänger glauben, Git speichere nur „Änderungslisten“. Das interne Modell ist jedoch näher an einem Satz von **Momentaufnahmen**. Das hilft beim Verständnis von Branches, Merges und Wiederherstellung.

Prinzip 3: Bewusstes Festschreiben von Änderungen

Git speichert nicht jede Zwischenbewegung automatisch. **Du** entscheidest, wann ein Stand wichtig genug ist.



Dieses bewusste Festschreiben zwingt dazu, Arbeitsschritte sinnvoll zu strukturieren. Gute Commits machen Projekte verständlicher und die Zusammenarbeit effizienter.

Prinzipien 4–6: Vergleichen, Verzweigen, Integrität

Änderungen vergleichen

Versionskontrolle ist auch ein **Vergleichswerkzeug**. Wenn ein Fehler auftaucht, lässt sich gezielt nachsehen, ab wann er entstanden ist und welche Änderung ihn verursacht hat.

Paralleles Arbeiten mit Branches

Git verwaltet parallele Entwicklungszweige (**Branches**). Änderungen sollen isoliert ausprobierbar und später integrierbar sein – ohne den stabilen Hauptstand zu gefährden.

Integrität & Nachvollziehbarkeit

Inhalte werden intern mit **Prüfsummen** identifiziert. Git erkennt Änderungen eindeutig; Manipulationen oder Beschädigungen fallen leichter auf. Die Git-Einführung hebt diese Integrität ausdrücklich hervor.

VERSIONSKONTROLLMODELLE

Zentralisierte Versionskontrolle

Bei der **zentralisierten Versionskontrolle** (z. B. SVN, CVS) liegt die offizielle Projektgeschichte auf **einem zentralen Server**. Entwickler arbeiten mit Arbeitskopien und kommunizieren ständig mit diesem Server.

Vorteile

- Klare „zentrale Wahrheit“
- Einfache Verwaltung & Rechtevergabe
- Konzeptionell leicht verständlich

Nachteile

- Serverausfall = kritischer Engpass
- Viele Operationen serverabhängig
- Lokale Kopie ohne vollständige Historie
- Weniger flexibel für moderne Workflows



VERSIONSKONTROLLMODELLE

Verteilte Versionskontrolle

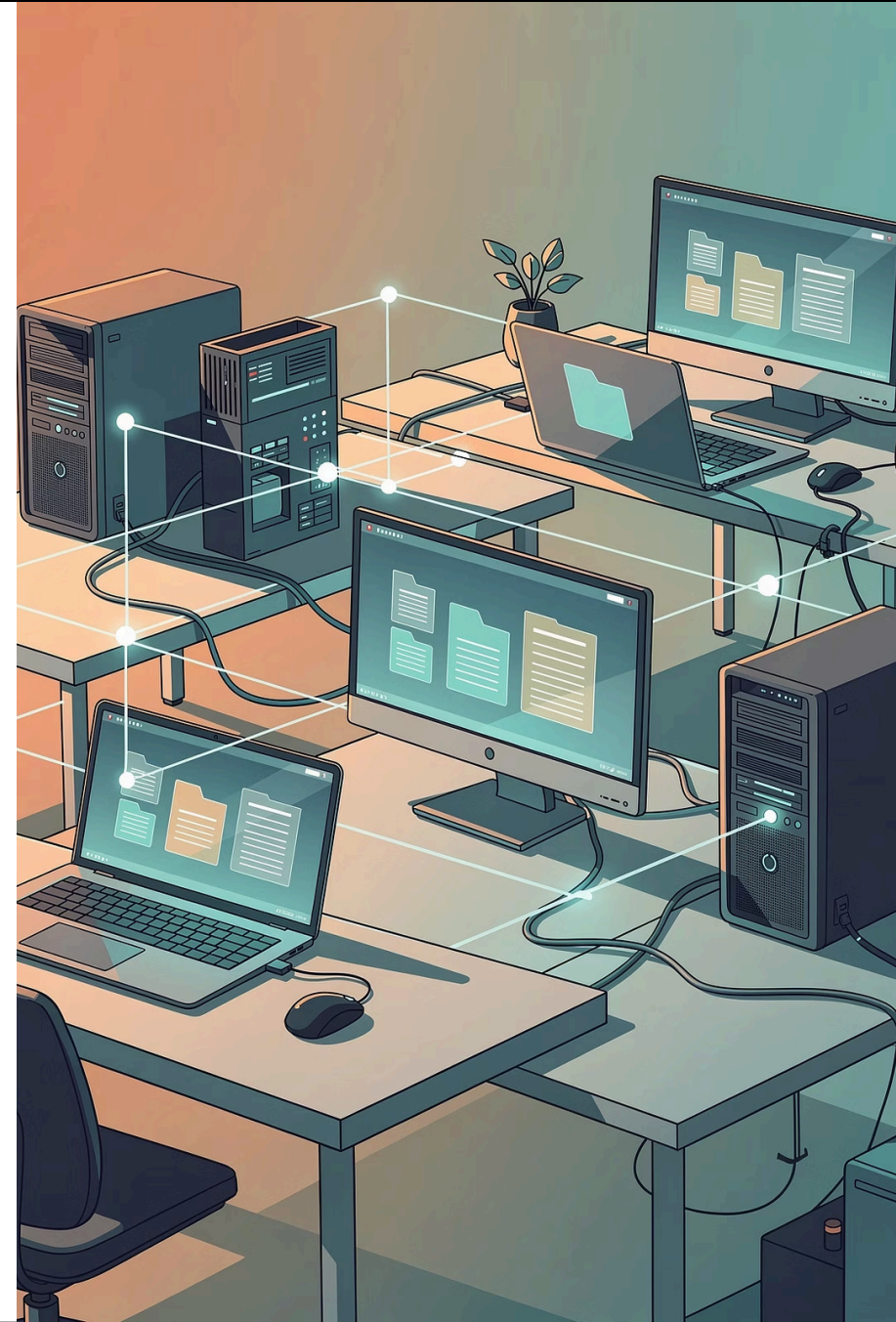
Bei der **verteilten Versionskontrolle** hat jeder Entwickler lokal eine **vollständige Kopie des Repositories inklusive Historie**. Git ist das bekannteste Beispiel. Auch bei Git gibt es im Team oft ein gemeinsames Remote-Repository (z. B. GitHub), aber technisch ist dieses Remote nicht die einzige vollständige Quelle.

Vorteile

- Viele Aktionen lokal & sehr schnell
- Arbeiten ohne Netzwerk möglich
- Jeder Klon = zusätzliche Sicherheitskopie
- Leistungsstarkes Branching & Merging

Nachteile

- Gedanklich anspruchsvoller am Anfang
- Begriffe wie push, pull, fetch, merge müssen sauber verstanden werden
- Ohne klares Teamvorgehen unübersichtlich



Direkter Vergleich: Zentralisiert vs. Verteilt

Aspekt	Zentralisiert	Verteilt
Projekt-Historie	Vor allem auf dem zentralen Server	Vollständig lokal vorhanden
Arbeiten ohne Netzwerk	Stark eingeschränkt	Gut möglich
Geschwindigkeit	Oft serverabhängig	Häufig sehr schnell lokal
Serverausfall	Kritischer Engpass	Weniger kritisch
Typische Systeme	SVN, CVS	Git, Mercurial
Branching & Merging	Oft umständlicher	Sehr stark unterstützt

 **Wichtigste Erkenntnis:** Git ist verteilt – das prägt fast alles, was man mit Git macht.

Nutzen von Versionskontrolle im Arbeitsalltag



Fehler rückgängig machen

Änderungen sauber zurückverfolgen – kein Durchsuchen alter ZIP-Dateien.
Menschen arbeiten mutiger, wenn Änderungen nachvollziehbar bleiben.



Zusammenarbeit ohne Überschreiben

Versionskontrolle führt Änderungen zusammen, macht Konflikte sichtbar und erzeugt eine gemeinsame Historie.



Dokumentation durch Projektgeschichte

Wann wurde ein Feature eingeführt? Wann ein Bug behoben? Warum wurde eine technische Entscheidung getroffen? Die Historie beantwortet das.



Weitere Vorteile im Alltag



Sicher experimentieren

Neue Ideen ausprobieren, ohne den stabilen Stand zu gefährden.
Versionskontrolle macht Experimentieren kontrollierbar.



Besser lernen & sauberer denken

Arbeit in nachvollziehbare Schritte zerlegen, Änderungen bewusst bündeln, Entscheidungen dokumentieren, eigene Fehler analysieren statt verstecken.



Nicht nur für Quellcode

Auch für Konfigurationsdateien, Infrastrukturdefinitionen, Dokumentation, Skripte und Markdown-Notizen sinnvoll – überall wo Inhalte sich entwickeln.

GIT-INSTALLATION

Git installieren & einrichten

Bevor du mit Git arbeitest, brauchst du zwei Dinge:

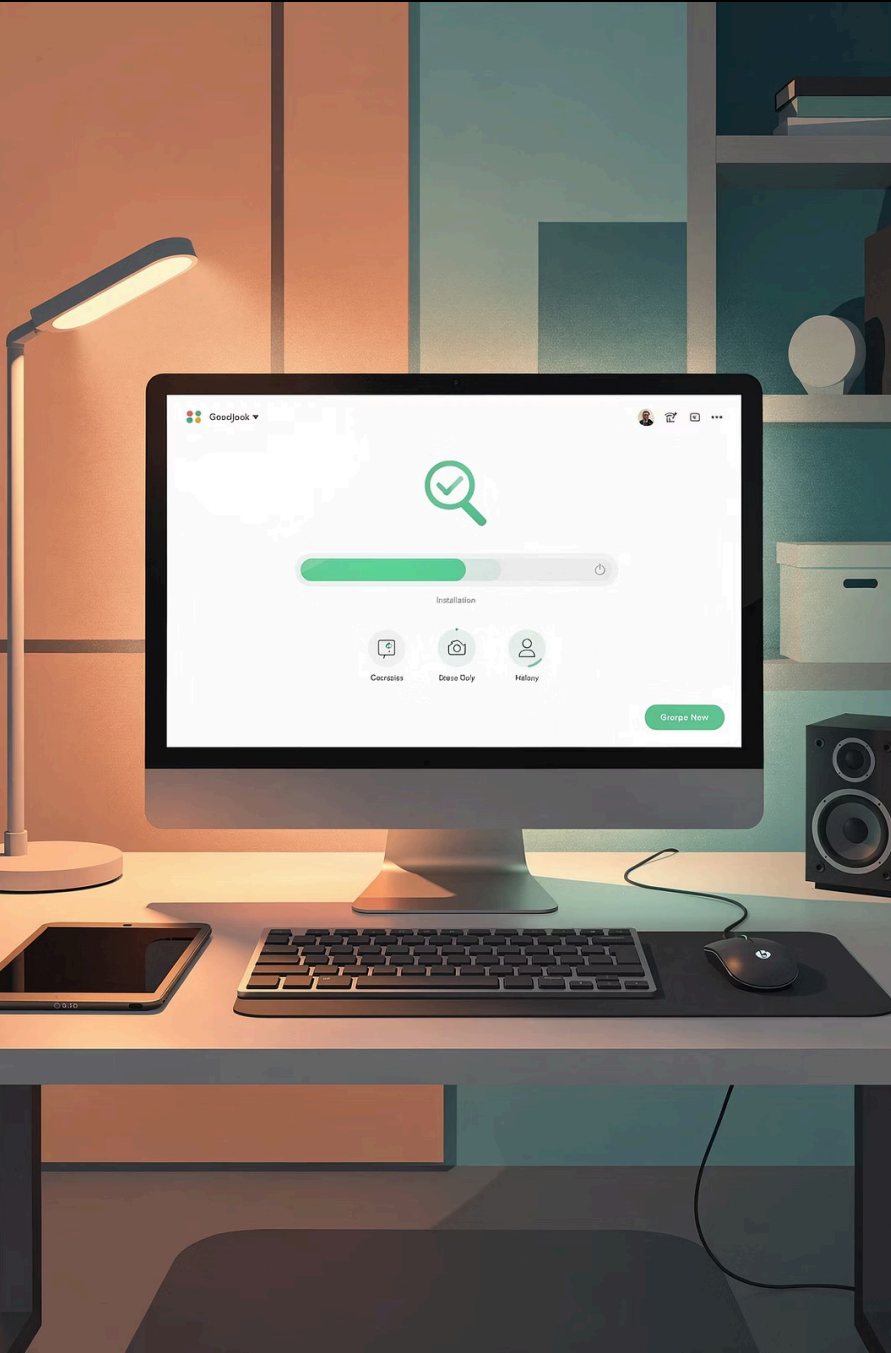
1. Git installieren

Git muss auf deinem System installiert sein – der Weg hängt vom Betriebssystem ab.

2. Git einrichten

Git sollte so konfiguriert werden, dass deine Commits korrekt deinen **Namen und deine E-Mail-Adresse** tragen.

Die offiziellen Installationswege werden auf der Git-Website und im Pro-Git-Buch beschrieben.



Git installieren: Windows, macOS & Linux

1

Windows

Offizieller Installer von **Git for Windows** über git-scm.com. Standardoptionen beibehalten, danach Git Bash oder Terminal öffnen.

2

macOS

Via **Homebrew**: `brew install git`, oder über Xcode Command Line Tools, oder direkt von der offiziellen Git-Quelle.

3

Linux

Über den Paketmanager: `sudo apt install git` (Debian/Ubuntu), `sudo dnf install git` (Fedora), `sudo pacman -S git` (Arch).

Auf allen Systemen gilt: Installation mit `git --version` prüfen. Eine Versionsnummer bestätigt die erfolgreiche Installation. Erscheint „command not found“, liegt meist ein Installations-, PATH- oder Neustartproblem vor.

Grundkonfiguration: user.name & user.email

Nach der Installation solltest du Git sagen, **wer du bist**. Diese Angaben landen in jedem Commit als Metadaten.

```
git config --global user.name "Max Mustermann"  
git config --global user.email "max@example.com"
```

Die drei Konfigurationsebenen

- `--system`: Gilt für alle Nutzer auf dem Rechner
- `--global`: Gilt für deinen Benutzer (empfohlen)
- `--local`: Gilt nur im aktuellen Repository

Konfiguration prüfen

Mit `git config --global --list` alle gesetzten Werte anzeigen. Mit `git config --list` die im aktuellen Repository aktiven Werte sehen.

Wichtig: `user.name` und `user.email` sind **Commit-Metadaten in Git selbst** – nicht automatisch ein GitHub-Login.

- ❏ **Typischer Denkfehler:** Viele glauben, user.name und user.email hätten automatisch etwas mit einem GitHub- oder GitLab-Login zu tun. Git ist das Versionskontrollsystem – GitHub ist nur ein Hosting-Dienst für Git-Repositories.

Git vs. GitHub: Ein wichtiger Unterschied

Git

Das **Versionskontrollsystem** selbst. Läuft lokal auf deinem Rechner. Verwaltet die Projekthistorie. Speichert Commit-Metadaten wie Name und E-Mail unabhängig von jeder Online-Plattform.

GitHub / GitLab / Bitbucket

Hosting-Dienste für Git-Repositories. Plattformen können Commits deinem Konto zuordnen, wenn die verwendete E-Mail-Adresse dort bekannt ist – aber das ist eine separate Funktion.

ERSTE SCHRITTE

Ein Repository initialisieren: git init

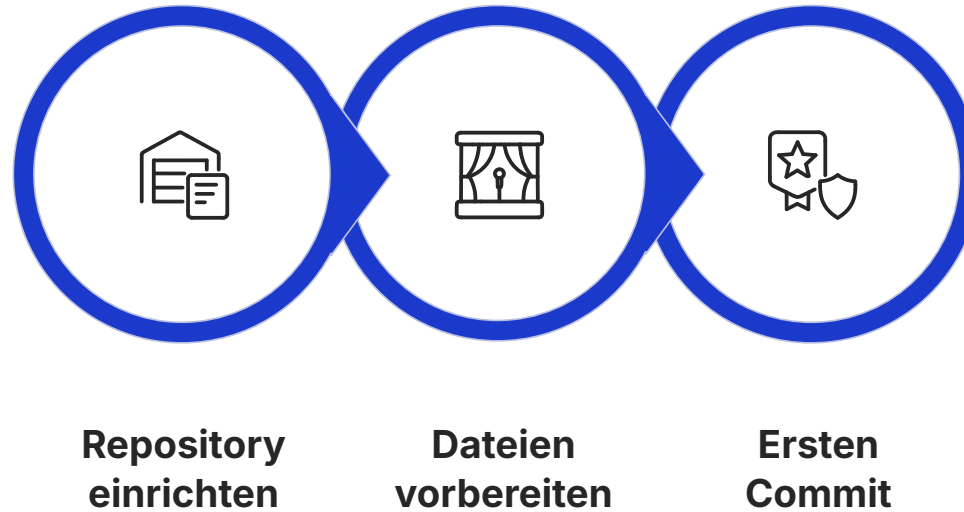
Ein **Repository** ist der Ort, an dem Git die Versionsgeschichte eines Projekts verwaltet. Mit `git init` sagst du Git, dass ein Ordner ab jetzt unter Versionskontrolle stehen soll.

```
mkdir mein-projekt  
cd mein-projekt  
git init
```

Git legt dabei einen versteckten `.git`-Ordner an, der Konfiguration, Branches, Objektdatenbank und Commit-Historie enthält. Deine eigentlichen Projektdateien bleiben unverändert. **Wichtig:** `git init` allein speichert noch keine Version deiner Arbeit – dafür braucht es einen ersten Commit.



Was git init macht – und was danach kommt



Bestehende Projekte nachträglich

Du kannst `git init` auch in einem Ordner ausführen, in dem bereits Dateien liegen. Git löscht oder verändert dabei nichts – es richtet nur die Versionskontrolle ein.

Bist du in einem Repository?

Mit `git status` prüfen: Bist du in einem Git-Repository, zeigt Git den aktuellen Zustand. Außerhalb erscheint: „not a git repository“.

Git-Hilfe: Selbsthilfe gezielt nutzen

Git bringt sehr gute eingebaute Hilfe mit. Professionelles Arbeiten bedeutet auch, Dokumentation zu nutzen statt nur Befehle auswendig zu lernen.

01

Allgemeine Hilfe

`git help` – allgemeine Übersicht. `git help -a` – Liste aller verfügbaren Unterbefehle.

03

Kurze Übersicht

`git init -h` – kompakte Kommandohilfe direkt im Terminal. Schneller als `--help`.

02

Hilfe zu einem Befehl

`git help init` oder `git init --help` – öffnet die vollständige Dokumentation zum Befehl.

04

Orientierung im Alltag

`git status` – die praktischste Selbsthilfeoption überhaupt. Zeigt Zustand, geänderte Dateien, Branch und vorgemerkte Änderungen.

Die goldene Faustregel bei Unsicherheit

1

git status

Was ist gerade der Zustand meines Repositories?

2

git diff

Was hat sich konkret geändert?

3

git log

Was ist in der Historie passiert?

Mit diesen drei Befehlen lassen sich schon sehr viele Situationen selbst verstehen. Wenn du einen unbekannten Befehl siehst: zuerst `git help <befehl>` – achte auf NAME, SYNOPSIS, DESCRIPTION und OPTIONS.

📌 **Prinzip:** Wenn du unsicher bist – zuerst `git status`, dann `git help <befehl>`, erst dann den Befehl ausführen.

Das mentale Grundmodell für den Start

Repository

Der verwaltete Projektkontext – `git init` macht einen Ordner dazu.

Snapshots & Commits

Git speichert Projektzustände als nachvollziehbare Momentaufnahmen in der Historie.

Commit-Metadaten

`user.name` und `user.email` gehören zu jedem Commit – global konfigurieren.

Orientierung

`git help` und `git status` sind zentrale Werkzeuge – immer zuerst nutzen.

Dieses Grundmodell ist die Basis für alle weiteren Themen: `add`, `commit`, `branch`, `merge`, `clone`, `push` und `pull` bauen direkt darauf auf.